NTS Pool Anycast Deployment Report

Running secure time at scale:

Operational observations from a BGP anycast NTS Pool testbed

'Secure Time for a Safe Internet' – Milestone 7
Supported by the ICANN Grant Program

Marco Davids & Giovane Moura, SIDN Labs

---

1 Introduction

Network Time Security (NTS) enhances the security of the Network Time Protocol (NTP) by providing cryptographic authentication and protection against on-path attacks.

While traditional NTP services are commonly deployed in pool configurations or via BGP anycast[1] to achieve global reach, scalability, and resilience, the introduction of NTS adds new operational and architectural challenges.

This report documents the findings of Milestone 7 of the broader NTS Pool experiment, specifically focusing on the operational implications of deploying NTS services using BGP anycast.

NTS introduces partially stateful elements such as TLS-based key establishment (NTS-KE) and subsequent cookie handling, which may interact in unexpected ways with BGP anycast routing. These characteristics raise questions about session continuity, failover behaviour, key distribution, and operational complexity in multi-node anycast deployments.

The goal of our study is to identify these challenges and evaluate practical approaches for operating NTS servers using BGP anycast in the context of an NTS Pool.

We evaluated how an NTS-secured time distribution system behaves when exposed to real-world BGP anycast dynamics.

The scope of this work is limited to operational and architectural aspects of running NTS anycast services, including deployment considerations, key management, failover behaviour, and client interaction.

As part of our study, an *anycasted* operational NTS Pool prototype was deployed and operated for a period of six months.

This report documents the selected architecture, deployment experience, observed behaviour, and potential pitfalls, and provides recommendations for running NTS anycast services in an NTS Pool environment.

2 Deployment Goals

The deployment of NTS servers via BGP anycast for this milestone aimed to achieve the following goals:

- **Global Availability** - Provide NTS-secured time services accessible from multiple regions behind a single anycast IP address (IPv4 and IPv6).
- **Operational Resilience** - Ensure failover behaviour and session continuity in the presence of node outages or routing changes.

[1] https://datatracker.ietf.org/doc/html/rfc8633#section-7

- **Practical NTS Pool Integration** - Evaluate how anycast deployment interacts with the pool model, including cookie distribution and key management.
- **Monitoring and Observability** - Collect operational metrics to assess performance, stability, and client behaviour over a six-month period.
- **Identification of Pitfalls** - Document challenges and lessons learned to inform future NTS anycast deployments.

3 About BGP Anycast

In a BGP anycast deployment, a single IP address is announced from multiple geographically distributed nodes; BGP routing generally directs clients to the topologically nearest node. All nodes in the BGP anycast testbed share the same IPv4 and IPv6 anycast address, hence forming a 'virtual' single Time Source, while remaining individually reachable via their unicast addresses for monitoring and management purposes.

Beyond the general challenges already known from BGP anycast deployments - such as routing fluctuations, load distribution, and node failover - Network Time Security (NTS) introduces additional complexities due to its stateful and cryptographic nature.

NTS key establishment relies on TLS sessions, which are stateful. Unlike traditional stateless anycast services, NTS deployments introduce operational dependencies between geographically distributed nodes due to shared cryptographic state. Clients must reach the same NTS-KE server node throughout the handshake. BGP anycast routing changes can redirect clients mid-handshake, resulting in failed session establishment or dropped requests.

We did not further investigate this effect in this milestone, as the study was limited to a single, unicast Key Exchange Load Balancer (KELB) used by time users for NTS-KE.

4 System Setup

The experiment was conducted on the SIDN Labs BGP anycast testbed[2], consisting of 32 globally distributed nodes running Flatcar Container Linux with tailored provisioning and custom-built Docker images for the experimental workloads.

The NTS Pool project consists of two approaches[3]: one based on an intermediate Key Exchange server and NTS extensions, as described in `draft-ietf-ntp-nts-keyexchange-pool`[4], introducing a KE Load Balancer (KELB) between clients (Time Users) and time servers (Time Sources), and another based on DNS SRV records[5] under de `_ntske._tcp` label.

[2] https://anycast.sidnlabs.nl/
[3] https://trifectatech.org/blog/enabling-pools-in-nts/
[4] https://datatracker.ietf.org/doc/draft-ietf-ntp-nts-keyexchange-pool/
[5] https://www.ietf.org/archive/id/draft-ladd-nts-for-ntp-pool-00.html

The main difference between the two approaches is that the KELB-based approach requires no changes to client software - only to server software - whereas the SRV-based approach requires changes to client software, but no modifications to server software.

For this pilot, a custom Docker image was deployed containing an adapted version of Chrony[6] to support the required NTS Pool functionality. The deployment exposed a stratum 2 NTS/NTP service.

The Time Source ("`anycast.ntspool.nl`") was registered with the NTS Pool[7] using its anycast address, allowing clients to be routed to the nearest available node. Consequently, it became available through the KELB pool as: "`[0123].ke.experimental.ntspooltest.org`" and through the SRV pool as "`srv.experimental.ntspooltest.org`".

Only the Time Source (TS) component was deployed in a BGP anycast configuration; the Key Exchange Load Balancer (KELB) remained unicast only and was excluded from BGP anycast. It was kept out of scope for this document, as it introduces additional dynamics left for future work (see par. 6).

4.1 Network Topology

Figure 1: Overview of anycast locations

The testbed is monitored using various methods, including BGPalerter, to detect routing anomalies.

[6] https://github.com/pendulum-project/chrony/releases
[7] https://experimental.ntspooltest.org/

4.2 Authentication Token

After a Time Source - itself an NTS time server - has been registered with the NTS Pool through the management dashboard, an authentication token is generated and made available via the dashboard. This token is used by the NTS Pool infrastructure to authenticate against the participating time server. Depending on the software in use, it must be configured to accept this token. For the adapted version of Chrony that was used, this meant replicating the token across all nodes and refer to it with the newly created '`ntsauthtokenfile`' option.

4.3 Shared Key Distribution

NTS uses cryptographic cookies to validate requests in a stateless manner. Each cookie is encrypted by the server using a master key, allowing any node that holds the same master key to decrypt it. In a BGP anycast deployment, cookies must be shareable across nodes, since a client can theoretically end up at any node. Consequently, all nodes need to share the same master key. We accomplished this by replicating an identical master key set across the platform without allowing the individual nodes to rotate them.

In our experimental setup, this was achieved by configuring Chrony with a single 'ntskeys' file replicated to all anycast instances while setting the '`ntsrotate`' option to 0. This ensures that each node operates with an identical key set and does not perform independent key rotation.

For controlled experimentation, this approach is sufficient and provides deterministic behaviour across the anycast deployment. However, such a model would not be suitable for production use, primarily because keys are never rolled. A production deployment would require a dedicated key management mechanism capable of securely rotating and automatically distributing keys across all anycast nodes in a way that does not disrupt the service. In particular, asynchronous key rotation between anycast nodes could lead to intermittent cookie validation failures during routing changes or failover events.

The experiment ran for approximately 6 months, during which several tests were conducted.

5 Observations

The NTS Pool infrastructure used during the experiment, which was operated from Europe, only had visibility of one single anycast node at a time, as was confirmed during testing. Under normal routing conditions, the health checks from this European pool monitor consistently landed on the Amsterdam anycast node due to topological proximity.

To observe failover behaviour, we simulated regional outages by successively disabling parts of the anycast network. This forced the KELB to establish sessions with distant

nodes. In the most extreme scenario, where only the node in Honolulu (USA) remained active, the round-trip time (RTT) for the NTS-KE handshake increased from approximately ~1ms to 254ms. During these tests, higher RTT paths also appeared to correlate with increased offset variation observed by the monitoring setup . This observed behaviour is likely related to increased network jitter and routing asymmetry inherent to trans-oceanic backhaul paths.

During the six-month deployment period, several additional operational observations were made:

- Under stable routing conditions, the BGP anycast deployment behaved largely transparently to NTS clients, without noticeable interoperability issues.
- No widespread compatibility problems were observed with existing NTS client implementations during the experiment.
- In practice, routing changes appeared to have limited impact on ongoing NTP traffic, likely because the NTS cookie mechanism allows requests to be validated by any node sharing the same key material. However, routing changes may still temporarily reduce synchronisation accuracy due to shifting network characteristics.
- Operational troubleshooting in an anycast environment proved significantly more complex than in traditional unicast deployments, particularly when attempting to correlate client behaviour with specific serving instances.
- The geographical diversity of the anycast nodes introduced substantial RTT variation between regions, which required operational monitoring components to be more tolerant of latency and timeout fluctuations.
- The experiment confirmed that observability and monitoring become increasingly important in globally distributed anycast deployments, especially when diagnosing regional routing anomalies or partial node failures.
- Additionally, operational monitoring highlighted that the large geographical distances (and resulting higher RTT) between the European-based KELB and distant anycast nodes initially caused timeout issues. This required a modification in the monitoring code[8] to handle these regional latency variations appropriately.

6 Conclusions and Future Work

BGP anycast is a well-known and proven technology for improving the global reach, scalability and resilience of network services. It can provide a pragmatic solution for routing clients to the nearest available NTS time server as an alternative to more advanced geolocation techniques. Our experiment demonstrated that operating a BGP anycasted Time Source as part of an NTS Pool is feasible.

However, transitioning this experimental model into a production environment introduces strict prerequisites. Most notably, the manual cryptographic key distribution

[8] https://github.com/pendulum-project/nts-pool/commit/4998e6aab301d9dddff35c136f689c16de70c4bd

and the suppression of key rotation used in this study are fundamentally unsuitable for production scale. A recommendation for any future operational deployment is the implementation of an automated, secure key management system. This system must be capable of dynamic key rotation and synchronous distribution across all distributed anycast nodes without causing operational downtime or disrupting active client cookie validation.

All 32 anycast nodes were monitored individually via their unicast addresses using a separate monitoring setup, independent of the NTS Pool monitoring. This monitoring was less comprehensive than the monitoring performed by the NTS Pool, but it allowed the correct operation of each node to be verified independently during the experiment.

While this was sufficient for an experimental deployment, a production deployment would require the NTS Pool itself to perform its more comprehensive monitoring from a significantly larger number of vantage points to provide adequate operational visibility across different regions.

During the experiment, we found that it would be beneficial for troubleshooting to identify which specific anycast instance generated a given response. This becomes particularly valuable in anycast environments where traffic engineering and routing behaviour can otherwise obscure the actual serving instance.

Similar functionality exists in the DNS ecosystem, for example via the NSID option (RFC5001) or the ‘ID.SERVER’ query (RFC4892). In NTP, this could potentially be achieved through an experimental NTP Extension Field (EF) exposing instance identifiers in responses. To avoid compatibility issues with legacy clients that do not support such extensions, this deterministic identification data should only be included in responses to explicitly enabled diagnostic requests.

Our experiments were conducted using unicast KELB and pool monitoring instances. Exploring the feasibility of deploying itself via BGP anycast could therefore be a valuable direction for future work.

Overall, the experiment suggests that BGP anycast can be a viable deployment model for large-scale NTS services, provided that operational complexity around key management, observability, and control-plane resilience is properly addressed.